Parallelizing the Head-Tracking Pipeline for Webcam-Based Fishtank VR

A SIMD, SIMT, and Pipelining Performance Study

CSC612M Integrating Project, Milestone 1 (Program Proposal)

Group members:
Reuchlin Charles Faustino
Gerard Christian Felipe
Kellie Kaw

GitHub repository: `https://github.com/kelliekaw/CSC612M-integrating-project`

0. Abstract

Fishtank VR gives a sense of depth on an ordinary monitor by tracking the viewer's head and redrawing the scene from the viewer's position. A long delay between head movement and the updated image can reduce this effect. This project builds a marker-based head-tracking pipeline and measures how much the compute time of its tracking stages changes when the techniques from this course are applied.

We implement the same pipeline three ways: a scalar C baseline, an x86-64 AVX2 assembly version called from C, and a CUDA version. We also write one small kernel in RISC-V assembly and analyze how it runs on a five-stage pipeline. We measure per-stage and end-to-end time with Nsight Systems and Nsight Compute, and we relate the results to the CPU performance equation, Amdahl's Law, and memory bandwidth. One question we want to answer is at what frame size the GPU becomes faster than AVX2 once the cost of copying data to the GPU is included.

The planned outputs are a head-tracked demo and a report of measured results. Webcam capture latency is measured separately and is not something this project tries to reduce.

1. Description of the Proposed Project

1a. Inputs

- **Live input:** frames from a USB webcam (640×480 and 1280×720, at 30 fps or higher). The capture stage converts them to 8-bit BGRA, so each pixel is 4 bytes and rows can be kept aligned for vector loads.
- **Tracked target:** a bright LED marker (green or infrared-pass) worn on a headband or glasses. A marker keeps the tracking step simple and easy to parallelize, so the project can stay focused on architecture and not on computer vision.
- **Benchmark input:** a recorded frame sequence and synthetic frames at several resolutions (320×240 up to 1920×1080) with a known marker path. Benchmarks run on these files so results can be repeated and do not depend on USB or driver timing. The synthetic frames also give ground truth for checking correctness.

1b. Proposed Process

The pipeline turns each frame into a head position that is used to set the render camera.

- **S0:** Webcam capture and format conversion. Not parallelized.

- **S1:** Marker score map (per-pixel color distance using saturating arithmetic). Parallelized with AVX2 and CUDA.
- **S2:** 3×3 separable blur to reduce noise. Parallelized with AVX2 and CUDA.
- **S3:** Threshold to a binary mask. Parallelized with AVX2 and CUDA.
- **S4:** Centroid from image moments (m_{00}, m_{10}, m_{01}), a parallel reduction. Parallelized with AVX2 and CUDA.
- **S5:** Pixel position to 3D head position (pinhole camera model). Not parallelized.
- **S6:** Smoothing and prediction filter (1 Euro filter). Not parallelized.
- **S7:** Off-axis projection and rendering of a simple 3D scene. Not parallelized.

i. What will be parallelized

Stages S1 to S4 will be parallelized with both SIMD and SIMT. They work on each pixel or a small neighborhood of pixels, and pixels do not depend on each other. The one exception is the final sum in S4, which is a reduction. This makes them a good fit for data-level parallelism. We expect these stages to account for most of the compute time in the tracking pipeline, and we will confirm this by measurement.

We will also build a fused version (S1 to S4 in a single pass over the image) for both targets. This lets us see how much of the cost comes from memory traffic and how much from arithmetic.

Separately, a small threshold-and-accumulate kernel will be written in RISC-V assembly to study instruction-level parallelism and pipeline hazards (CLO5).

ii. Related work and how this project differs

- Fishtank VR was described by Ware, Arthur, and Booth (1993), who studied head-coupled perspective on a regular monitor. Lee (2008) described a low-cost head-tracking setup using infrared LEDs and a Wii Remote. Kooima (2009) described a general method for off-axis projection.
- OpenCV (Bradski, 2000) includes vectorized and CUDA-accelerated image operations, and Intel IPP and NVIDIA NPP provide optimized building blocks. These libraries are general purpose. They are not written for this specific pipeline, and they hide data layout and memory transfer choices that this course covers.
- Latency in head-tracked displays is often reduced with prediction filters such as the 1 Euro filter (Casiez, Roussel, and Vogel, 2012). Prediction hides delay through an algorithm. This project instead measures the compute time of the tracking stages, and reports that separately from any prediction effect.

Our implementation differs from these in three ways. First, we write the kernels ourselves (AVX2 in assembly and CUDA C) instead of using library functions, so we control data layout and memory access. Second, we run the same pipeline on scalar, SIMD, and SIMT targets with the same inputs and outputs, so the results can be compared. Third, we report the cost of copying data to the GPU as its own result and look for the frame size at which the GPU becomes faster than AVX2. We do not assume in advance that it will be.

iii. Plan for parallelization

AVX2 (x86-64 assembly with NASM, called from C):

- Process 32 bytes per vector (8 BGRA pixels) using saturating byte instructions such as `vpsubusb` and `vpmaxub` for the score map.
- Implement the blur as a horizontal pass followed by a vertical pass, processing a few rows at a time so that data stays in cache.
- Compare pixels against the threshold with vector compare instructions and accumulate m_{00}, m_{10}, and m_{01} in vector registers, then add the lanes together once at the end.
- Use aligned buffers, handle leftover pixels at the end of each row with scalar code, and compare an unfused and a fused version.

CUDA (SIMT):

- Use one thread per pixel for S1 and S3, shared-memory tiles with extra border pixels for the blur, and a warp-shuffle plus block-level reduction for the moments.
- Test pinned host memory, asynchronous copies, and two CUDA streams, to see whether copying frame $N+1$ can overlap with computing frame N.
- Compare separate kernels against one fused kernel, and test several block sizes.

RISC-V (instruction-level parallelism, CLO5):

- Write the threshold-and-accumulate loop in RISC-V assembly and run it in RARS.
- Identify structural, data (including load-use), and control hazards on the five-stage pipeline, and count the stall cycles they cause.
- Write a rescheduled and unrolled version, and compare instruction count, estimated CPI, and total cycles using the CPU performance equation.

Correctness and measurement:

- The scalar C version is the reference. The integer kernels (S1 to S3) should match it exactly, and the S4 centroid should match within a small tolerance on the synthetic data.
- Timing uses high-resolution timers around each stage, with warm-up runs and many repetitions, and we report the mean and the spread. Nsight Systems shows the timeline and whether transfers overlap with compute. Nsight Compute shows occupancy, memory throughput, and whether each kernel is limited by memory or compute.
- Analysis uses the CPU performance equation, Amdahl's Law (expected versus measured end-to-end speedup), and achieved memory bandwidth compared with the hardware's peak.

iv. Processes that will not be parallelized

- **S0 (capture and conversion)** depends on the camera, USB, and driver. Its latency is measured separately and reported next to the results. It may set a minimum on end-to-end delay that this project cannot lower.
- **S5 and S6** work on a single point per frame and should take very little time, so parallelizing them would add overhead without a clear benefit. They stay scalar and are counted as the serial part in the Amdahl's Law analysis.
- **S7 (rendering)** is a simple demo layer built with a standard graphics API. It is there to show the tracking result and is not an optimization target.

1c. Outputs

- A head-tracked fishtank VR demo in which the scene's perspective follows the marker.
- Three tracker implementations (scalar C, AVX2 assembly, CUDA) that produce the same marker coordinates.
- A RISC-V assembly kernel with a hazard and CPI analysis.
- Benchmark data: per-stage and end-to-end time, speedup, and frames per second across resolutions, including the frame size at which the fastest implementation changes, if there is one.
- Profiling results (Nsight Systems and Nsight Compute screenshots and metrics) and an Amdahl's Law analysis.
- A final report and demo, with source code and benchmark scripts in the public GitHub repository.

Planned success criteria. The AVX2 and CUDA versions are checked against the scalar reference. A measured speedup is reported for stages S1 to S4 on each target, including cases where the GPU is slower on small frames. Each result is explained using profiler data.

2. Proposed Target Implementation Platform

- **OS and toolchain:** Windows with Visual Studio 2026 (C/C++), NASM for x86-64 AVX2 assembly, and the NVIDIA CUDA Toolkit.
- **CPU:** x86-64 processor with AVX2 support: [model]. AVX-512 is optional and will be used only if the CPU supports it.
- **GPU:** NVIDIA GPU with CUDA support: [model, compute capability, memory bandwidth].
- **Camera:** USB webcam, 720p at 30 fps or higher: [model]. Capture uses a standard camera API and is not part of the optimization.
- **Profiling:** NVIDIA Nsight Systems and Nsight Compute, plus high-resolution CPU timers.
- **RISC-V:** RARS simulator for the assembly kernel and the hazard analysis.
- **Display:** a standard monitor, with a basic OpenGL off-axis projection scene.

3. Mid-Project Milestones

The weeks below will be adjusted to match the course calendar.

1. **Week 6, baseline and test setup:** webcam capture working, recorded and synthetic datasets prepared, scalar C pipeline finished, timing code and correctness checker in place.
2. **Week 8, SIMD and SIMT kernels:** AVX2 kernels for S1 to S4 checked against the baseline, unfused CUDA kernels checked, and first benchmark numbers collected.
3. **Week 10, optimization and RISC-V:** fused AVX2 and CUDA versions, pinned memory and streams, and the RISC-V kernel with a first hazard analysis.
4. **Weeks 11 to 12, profiling and integration:** Nsight analysis, resolution sweep, Amdahl's Law analysis, and integration with the live webcam and render demo.
5. **Week 13, final deliverables:** end-to-end latency measurements, final report, and project demo.

4. References

- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- Casiez, G., Roussel, N., & Vogel, D. (2012). 1 Euro filter: A simple speed-based low-pass filter for noisy input in interactive systems. *Proceedings of CHI 2012*, 2527-2530.
- Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- Hennessy, J. L., & Patterson, D. A. (2018). *Computer Organization and Design: A Hardware/Software Approach* (RISC-V Edition). Morgan Kaufmann.
- Intel Corporation (2026). *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volumes 1 and 2.
- Kirk, D., & Hwu, W.-M. (2017). *Programming Massively Parallel Processors*. Morgan Kaufmann.
- Kooima, R. (2009). *Generalized perspective projection*. Technical report, Louisiana State University.
- Lee, J. C. (2008). Hacking the Nintendo Wii Remote. *IEEE Pervasive Computing*, 7(3), 39-45.
- RISC-V Organization (2026). *RISC-V Instruction Set Manual, Volume 1: Unprivileged Specification*.
- Ware, C., Arthur, K., & Booth, K. S. (1993). Fish tank virtual reality. *Proceedings of INTERCHI '93*, 37-42.

AI Usage Declaration

Per the course AI usage policy, we declare the following: Claude Sonnet 5.5 was used to search references, with the prompt "Our group for our advanced computer architecture class research project requirement is planning on webcam based fishtank vr particularly on optimizing head tracking latency. Can you give us the most related literature and areas for novelty?" Claude Sonnet 5.5 was also used to draft and rewrite the text of this proposal and to format it in LaTeX, based on the course syllabus and the milestone outline we provided. No AI tools were used for code generation, error handling, or debugging.